

Q.NO :10

Question description

Umesh is an DS expert training youngsters struggling in DS to make them better.

Umesh usually gives interesting problems to the youngsters to make them love the DS.

One such day Umesh provided to the youngsters to solve the task such that, Reverse a Queue, Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Youngsters were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

- INITIALIZE FRONT AND REAR = -1
- CALL ENQUEUE FUNCTION WHILE(I<GIVEN SIZE OF QUEUE)
- CALL REVERSE
- MAKE A FOR LOOP(I=FRONT,J=REAR;I<J;I++;J++)
- SWAP(FRONT,REAR)

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

first line indicates the enqueue of each elements

last line indicates the reversed queue elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
5 6 3 1 2

EXPECTED OUTPUT

Queue:5 6 3 1 2
Reversed Queue:2 1 3 6 5

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

Queue:1 8 6 4 9 5 3 5
Reversed Queue:5 3 5 9 4 6 8 1

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int data,int l)

Test Case 4

KEYWORD

reverse();

Test Case 2

KEYWORD

void reverse()

Test Case 3

KEYWORD

enqueue(t,n);

Code:

```
#include <stdio.h>

int queue[100];
int front = -1, rear = -1;

void enqueue(int data, int l)
{
    if (rear == l - 1)
        return;
```

```
if (front == -1)
front = 0;

rear++;
queue[rear] = data;
}

void reverse()
{
int i, j, temp;

for (i = front, j = rear; i < j; i++, j--)
{
temp = queue[i];
queue[i] = queue[j];
queue[j] = temp;
}
}

int main()
{
int n, t;

scanf("%d", &n);

for (int i = 0; i < n; i++)
{
scanf("%d", &t);
enqueue(t,n);
}

printf("Queue:");
for (int i = front; i <= rear; i++)
printf("%d ", queue[i]);

reverse();

printf("\nReversed Queue:");
for (int i = front; i <= rear; i++)
printf("%d ", queue[i]);

return 0;
}
```

Output:

Test_case 1:

```
5
5 6 3 1 2
Queue:5 6 3 1 2
Reversed Queue:2 1 3 6 5 %
```

Test_case 2:

```
8
1 8 6 4 9 5 3 5
Queue:1 8 6 4 9 5 3 5
Reversed Queue:5 3 5 9 4 6 8 1 %
```